CM 3321 - Logic Programming and Artificial Cognitive Systems

Assignment 02

Card Transaction Authorization Advisor

Wimalasekera K.D.I.S.D.S. - 224219H

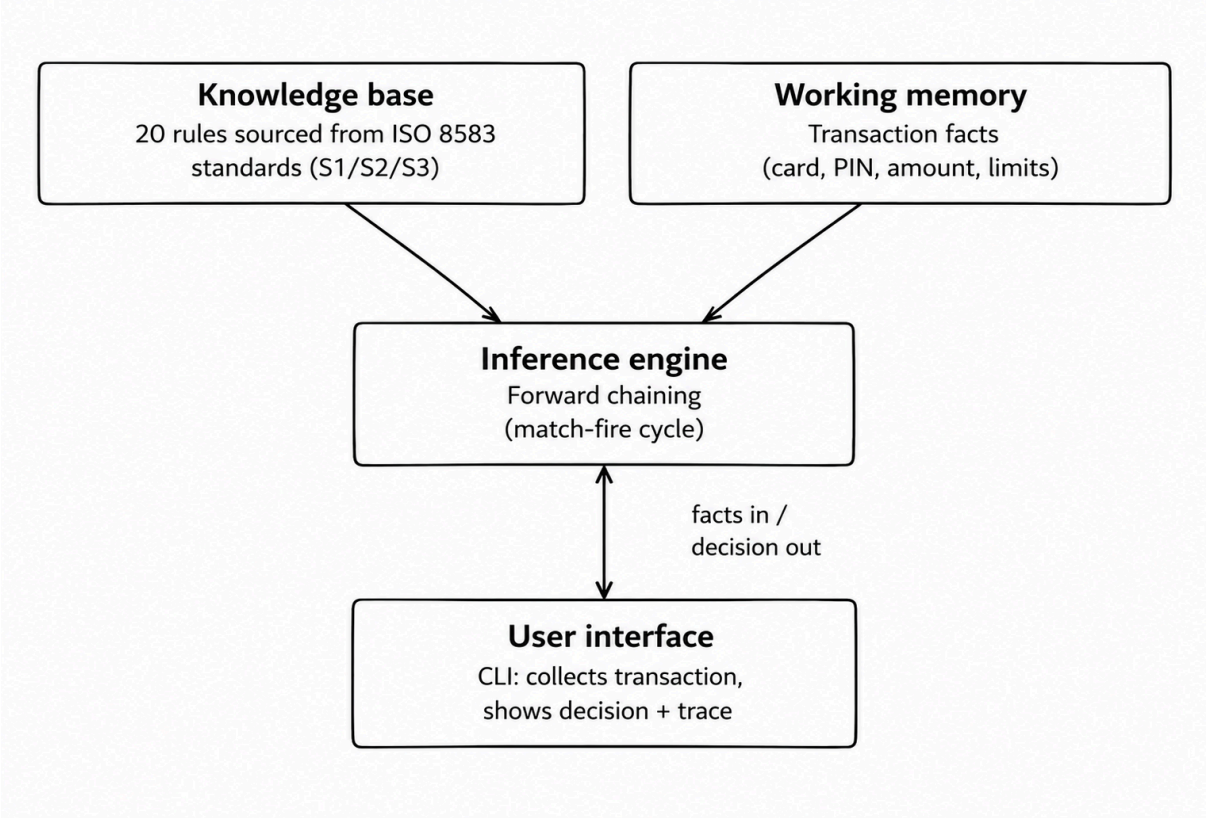
1. Introduction and Domain Selection

This report documents an expert system built for a real, narrow decision problem in card payment processing: given the details of a point-of-sale purchase or ATM withdrawal attempt, decide whether the transaction should be approved or declined, and which ISO 8583 (field 39) response code applies to the outcome. This domain was chosen because it maps directly onto standardised, publicly documented decision rules used across the card payment industry, rather than an internally invented policy - which makes the knowledge base straightforward to source and cite, as required for this assignment.

The system's knowledge base was compiled from three published technical references documenting the ISO 8583 standard's response codes and common card-scheme decline reasons.

2. Expert System Architecture

The system follows the classical expert system architecture: a static knowledge base of production rules, a working memory holding the current facts, a forward-chaining inference engine that matches rules against facts and fires them, and a user interface that supplies facts and displays the resulting decision together with an explanation trace.



Knowledge base - 20 IF-THEN production rules, each carrying its own source citation (Section 6, Annex A).

Working memory - a Python dictionary holding the transaction's input facts (card status, PIN details, amount, limits, etc.) plus every conclusion derived so far.

Inference engine - implements forward chaining: on each pass it scans every rule in the knowledge base and fires any whose condition is newly satisfied, repeating until a pass produces no new facts (a fixed point). This is the same match-fire-cycle strategy used by rule engines such as CLIPS or Jess, implemented here from scratch in plain Python so every step is visible for demonstration.

Explanation facility - every fired rule is appended to a trace list, so the interface can show exactly which rules fired, in what order, and from which source - not just the final decision.

User interface - a command-line interface that asks the operator for the transaction's details and displays the decision and trace.

3. User Interface

The interface is a command-line program (interface.py). It prompts the operator for the transaction's facts one at a time: card status, expiry, fraud flag, transaction type, whether the card is physically present, PIN details or CVV match, merchant category, terminal permission, amount, available balance, and (for withdrawals) the daily withdrawal limits already used. Once all facts are collected, they are passed to the inference engine as working memory, and the resulting decision code, message and explanation trace are printed. A graphical interface (e.g. a small Tkinter or web form) could be substituted for interface.py without changing the knowledge base or inference engine, since the two are cleanly separated.

4. Knowledge Base and Inference Engine

The knowledge base (knowledge_base.py) defines 20 rules as Rule objects, each with a unique id, a plain-language description, a condition function that reads working memory, an action function that writes new facts when the condition is true, and a source citation string. Rules R1-R17 test one condition each (card status, expiry, fraud flag, PIN, CVV, merchant category, amount validity, withdrawal limits, balance, function/terminal permission, timeout) and, if true, assign an ISO 8583 response code and reason. Rule ordering encodes priority: card-status hard stops (lost/stolen/blocked) are checked first, so a lost card is never reported as, say, an amount error. Rules R18-R20 are combination rules: R18 approves the transaction

(code 00) once none of R1-R17 has matched; R19 tags a risk tier for reporting purposes only; R20 compiles the human-readable message. R18-R20 are the author's own control logic, not card-scheme citations, and are labelled as such throughout.

The inference engine (inference_engine.py) implements forward chaining: run_inference() repeatedly scans the rule list and calls each rule's try_fire(), which fires the rule (applies its action and records it in the explanation trace) only if the condition holds and the rule has not already fired for this transaction. The loop stops once a full pass fires no new rule, i.e. the working memory has reached a fixed point.

5. Test Cases

The 14 test cases below (test_cases.py) were run automatically; all 14 produced the expected ISO 8583 code, confirming the rule priority ordering and the approve-by-default logic both behave correctly.

#	Scenario	Expected	Actual	Result
TC1	Straightforward approved purchase	00	00	**PASS**
TC2	Lost card presented	41	41	**PASS**
TC3	Stolen card presented	43	43	**PASS**
TC4	Blocked/restricted card	62	62	**PASS**
TC5	Expired card	54	54	**PASS**
TC6	Fraud engine flagged the transaction	59	59	**PASS**
TC7	PIN tries exhausted	75	75	**PASS**
TC8	Incorrect PIN entered (tries remaining)	55	55	**PASS**
TC9	Card-not-present, CVV mismatch	05	05	**PASS**
TC10	Merchant category restricted	03	03	**PASS**
TC11	Insufficient funds	51	51	**PASS**
TC12	ATM withdrawal exceeds daily amount limit	61	61	**PASS**
TC13	ATM withdrawal exceeds daily count limit	65	65	**PASS**
TC14	Valid ATM withdrawal within all limits	00	00	**PASS**

6. References (Source of Domain Knowledge)

[S1] myPOS Developer Documentation, "229 Response Codes" (ISO 8583 field 39 response code table). https://developers.mypos.com/en/doc/online_payments/v1_4/229-response-codes

[S2] softwareag Tech Community forum, "ISO8583 Message Format Support" (quoting the VISA Base I response code list). https://tech.forums.softwareag.com/t/iso8583-message-format-support/155110

[S3] CCBill Knowledge Base, "A Comprehensive List of Credit Card Decline Codes". https://ccbill.com/kb/?p=3602

Annex A: Complete Rules List (20 Rules)

ID	Rule (condition -> conclusion)	Source
R1	IF card_status = lost THEN code = 41 (pick-up, lost card)	S1 / S2
R2	IF card_status = stolen THEN code = 43 (pick-up, stolen card)	S1 / S2
R3	IF card_status = blocked THEN code = 62 (restricted card)	S1
R4	IF card is expired THEN code = 54 (expired card)	S1 / S2
R5	IF suspected_fraud = true THEN code = 59 (suspected fraud)	S3
R6	IF pin_tries_remaining <= 0 THEN code = 75 (PIN tries exceeded)	S3
R7	IF PIN required AND entered incorrectly (tries remain) THEN code = 55	S1 / S3
R8	IF card not present AND CVV mismatch THEN code = 05 (do not honor)	S1
R9	IF merchant category not allowed THEN code = 03 (invalid merchant)	S1
R10	IF amount <= 0 THEN code = 13 (invalid amount)	S1 / S2

R11	IF transaction type unrecognised THEN code = 12 (invalid transaction)	S1
R12	IF withdrawal would exceed daily amount limit THEN code = 61	S1 / S3
R13	IF daily withdrawal count limit reached THEN code = 65	S1
R14	IF amount > available balance THEN code = 51 (insufficient funds)	S1 / S2
R15	IF card does not support requested function THEN code = 40	S3
R16	IF terminal/acquirer not permitted THEN code = 58	S1 / S2
R17	IF host response timed out THEN code = 68 (response too late)	S3
R18	IF no decline rule (R1-R17) matched THEN code = 00 (approved)	System design
R19	IF approved THEN tag risk_tier from amount/balance ratio	System design
R20	Compile final message string from code + reason	System design

Annex B: Screenshots

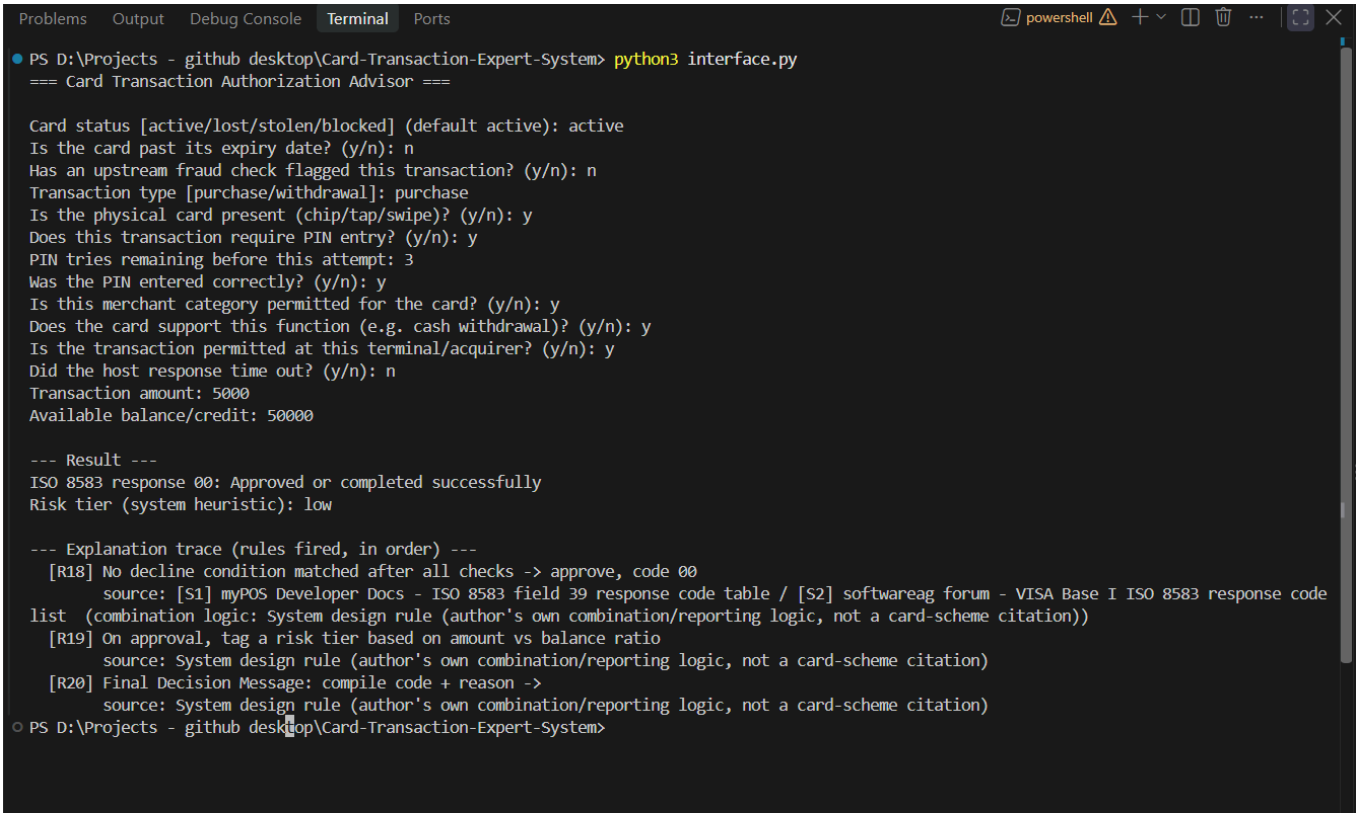


Figure 1: User Interface

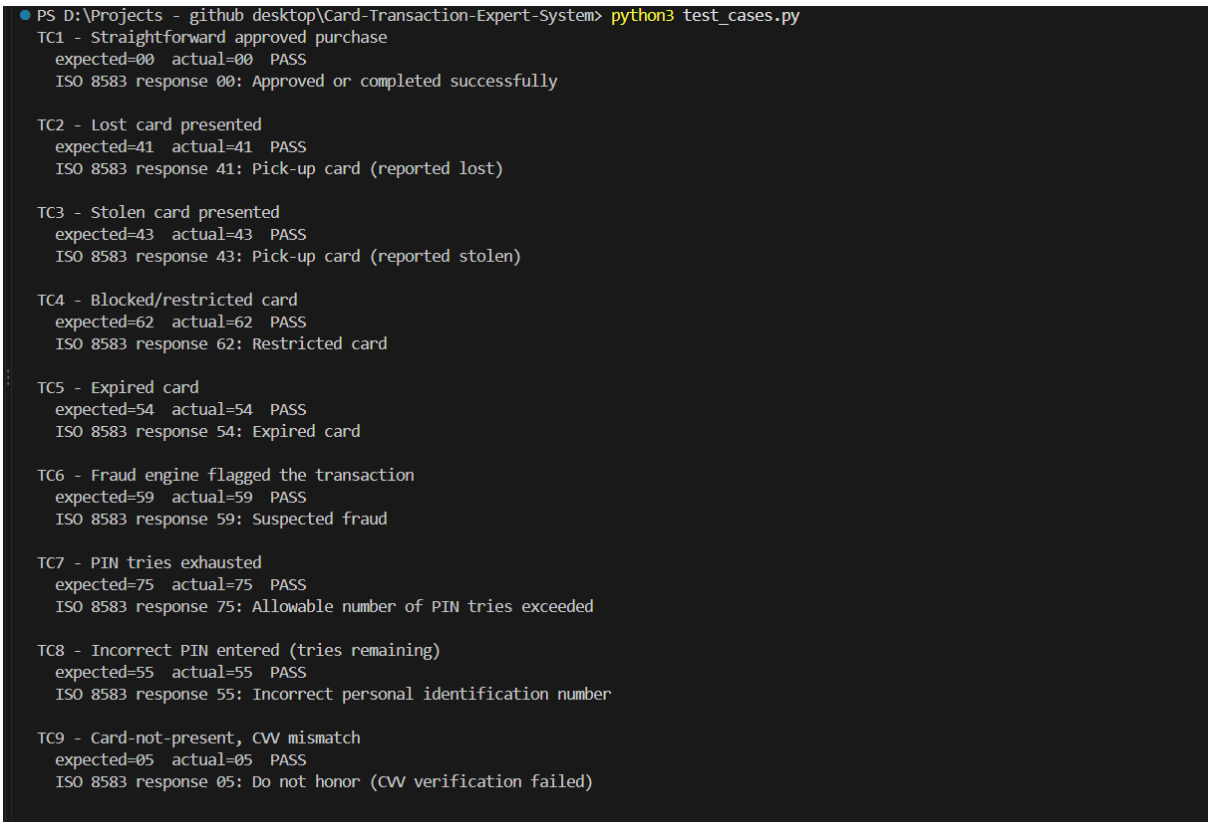


Figure 2: Test Cases - 1

```
TC9 - Card-not-present, CVV mismatch
  expected=05  actual=05  PASS
  ISO 8583 response 05: Do not honor (CVV verification failed)

TC10 - Merchant category restricted
  expected=03  actual=03  PASS
  ISO 8583 response 03: Invalid merchant

TC11 - Insufficient funds
  expected=51  actual=51  PASS
  ISO 8583 response 51: Insufficient funds

TC12 - ATM withdrawal exceeds daily amount limit
  expected=61  actual=61  PASS
  ISO 8583 response 61: Exceeds withdrawal amount limit

TC13 - ATM withdrawal exceeds daily count limit
  expected=65  actual=65  PASS
  ISO 8583 response 65: Exceeds withdrawal count limit

TC14 - Valid ATM withdrawal within all limits
  expected=00  actual=00  PASS
  ISO 8583 response 00: Approved or completed successfully

14/14 test cases passed.
PS D:\Projects - github desktop\Card-Transaction-Expert-System>
```

Figure 3: Test Cases - 2